

POLITECHNIKA ŁÓDZKA

WYDZIAŁ FIZYKI TECHNICZNEJ, INFORMATYKI I
MATEMATYKI STOSOWANEJ

Kierunek: Matematyka stosowana

Szeregowanie zadań

Oliwia Warda 263327

Łódź, czerwiec 2026

1 Wprowadzenie do problemu

Problem **szeregowania zadań** polega na wyznaczeniu odpowiedniej kolejności wykonywania zadań przy uwzględnieniu zależności występujących pomiędzy nimi.

W wielu rzeczywistych problemach nie wszystkie zadania mogą być wykonywane w dowolnej kolejności. Często wykonanie jednego zadania wymaga wcześniejszego zakończenia innego zadania. Powstaje wówczas sieć zależności, którą można przedstawić przy pomocy grafu skierowanego.

W takim modelu **wierzchołki** reprezentują zadania, a **krawędzie** reprezentują zależności pomiędzy zadaniami.

Jeżeli graf nie zawiera cykli, nazywany jest skierowanym grafem acyklicznym (**DAG** – **D**irected **A**cycl**I**c **G**raph).

Dla grafów DAG możliwe jest wyznaczenie poprawnej kolejności wykonywania wszystkich zadań. Problem ten rozwiązuje się przy pomocy sortowania topologicznego.

Sortowanie topologiczne polega na takim uporządkowaniu wierzchołków grafu, aby dla każdej krawędzi skierowanej (u, v) wierzchołek u występował wcześniej niż wierzchołek v .

2 Graf cykliczny i acykliczny

2.1 Graf acykliczny

Graf acykliczny nie zawiera żadnego cyklu.

Przykład:

$$A \rightarrow B \rightarrow C$$

Możliwe jest wyznaczenie kolejności:

$$A, B, C$$

Dlatego taki graf może zostać wykorzystany do modelowania procesu wykonywania zadań.

2.2 Graf cykliczny

Graf cykliczny zawiera przynajmniej jeden cykl.

Przykład:

$$A \rightarrow B \rightarrow C \rightarrow A$$

Powstaje sytuacja, w której zadanie A oczekuje na C, C oczekuje na B, a B oczekuje na A. Nie istnieje więc zadanie, od którego można rozpocząć pracę.

Z tego powodu sortowanie topologiczne jest możliwe wyłącznie dla grafów acyklicznych.

3 Zastosowania

Problem szeregowania zadań znajduje zastosowanie w wielu dziedzinach.

- Zarządzanie projektami
- Systemy operacyjne
- Produkcja przemysłowa
- Planowanie zajęć

i wiele innych.

4 Rozwiązanie problemu

Do klasycznych metod szeregowania zadań należą **algorytm DFS** oraz **algorytm Kahna**.

Algorytm DFS to klasyczna metoda rozwiązywania problemu sortowania topologicznego w grafach skierowanych acyklicznych (DAG). Umożliwia ona ustalenie poprawnej kolejności wykonywania zadań, które posiadają zależności typu „zadanie A musi być wykonane przed zadaniem B”. Algorytm DFS bada najgłębsze ścieżki w grafie, odkładając odwiedzone wierzchołki na specjalny stos. **Algorytm Kahna** to metoda dla grafów DAG, która umożliwia ułożenie zadań w liniowej kolejności od najwcześniejszych do najpóźniejszych, uwzględniając ich zależności (poprzedników). Działa w oparciu o analizę stopni wejściowych wierzchołków.

Jeżeli po zakończeniu działania algorytmu liczba odwiedzonych wierzchołków jest mniejsza od liczby wszystkich wierzchołków, oznacza to występowanie cyklu.

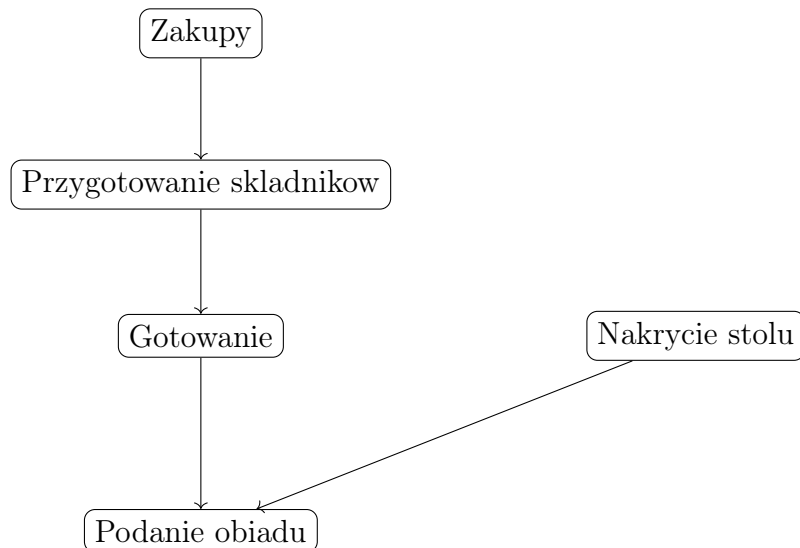
Złożoność czasowa algorytmów wynosi:

$$O(V + E), \text{ gdzie}$$

V – liczba wierzchołków, E – liczba krawędzi.

5 Przykład 1

Rozważmy proces przygotowania obiadu. Niektóre czynności muszą zostać wykonane wcześniej, zanim możliwe będzie wykonanie kolejnych. Zależności pomiędzy zadaniami przedstawia graf:



6 Implementacja w Pythonie

Do reprezentacji grafu wykorzystamy klasę **Graph**. Wierzchołki grafu odpowiadają zadaniam, natomiast krawędzie reprezentują zależności pomiędzy nimi. Wewnątrz klasy zaimplementowano dwa najpopularniejsze algorytmy sortowania topologicznego: **algorytm Kahna** oraz **algorytm DFS**.

```
from collections import deque

class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, u, v):

        self.add_vertex(u)
        self.add_vertex(v)
        self.graph[u].append(v)
```

```

def topological_sort_kahn(self):
    in_degree = {vertex: 0 for vertex in self.graph}

    for vertex in self.graph:
        for neighbor in self.graph[vertex]:
            in_degree[neighbor] += 1

    queue = deque([v for v in in_degree if in_degree[v] ==
0])

    result = []

    while queue:
        current = queue.popleft()
        result.append(current)

        for neighbor in self.graph[current]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(result) != len(self.graph):
        return "Graf zawiera cykl! Nie mozna ulorzyc
harmonogramu."

    return result

def topological_sort_dfs(self):
    visited = set()
    stack = []

    def dfs(vertex):
        visited.add(vertex)
        for neighbor in self.graph.get(vertex, []):
            if neighbor not in visited:
                dfs(neighbor)
        stack.append(vertex)

    for vertex in self.graph:
        if vertex not in visited:

```

```
dfs(vertex)
```

```
return stack[::-1]
```

Następnie tworzymy i budujemy graf opisujący proces przygotowania obiadu, dodając odpowiednie zależności.

```
g = Graph()
g.add_edge("Zakupy", "Przygotowanie_skladnikow")
g.add_edge("Przygotowanie_skladnikow", "Gotowanie")
g.add_edge("Gotowanie", "Podanieobiadu")
g.add_edge("Nakrycie_stolu", "Podanieobiadu")

print("Kolejnosc wykonywania zadan (Algorytm Kahna):")
for task in g.topological_sort_kahn():
    print(f"- {task}")

print("\nKolejnosc wykonywania zadan (Algorytm DFS):")
for task in g.topological_sort_dfs():
    print(f"- {task}")
```

Wynik działania programu:

Kolejnosc wykonywania zadan (Algorytm Kahna):

- Zakupy
- Nakrycie_stolu
- Przygotowanie_skladnikow
- Gotowanie
- Podanieobiadu

Kolejnosc wykonywania zadan (Algorytm DFS):

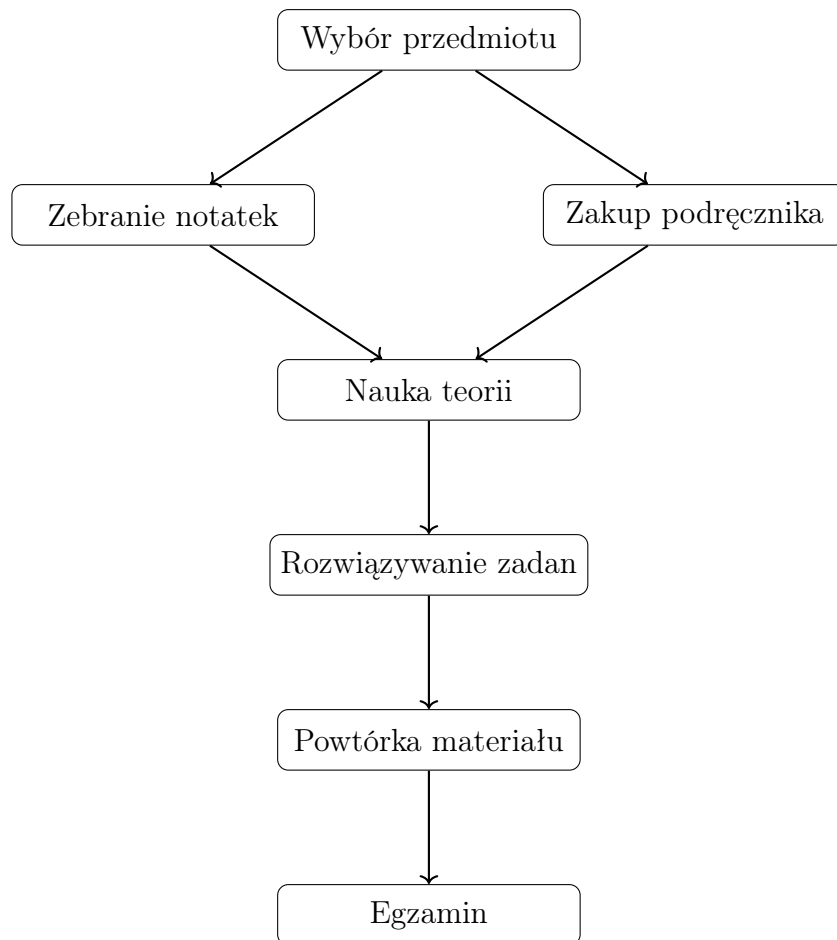
- Nakrycie_stolu
- Zakupy
- Przygotowanie_skladnikow
- Gotowanie
- Podanieobiadu

Jak widać, oba algorytmy zwróciły poprawne uszeregowanie zadań. Różnica polega jedynie na momencie wykonania zadania „Nakrycie stołu”, które w obydwu przypadkach zostało zaplanowane prawidłowo przed zadaniem „Podanie obiadu”.

7 Przykład 2

Kolejnym praktycznym zastosowaniem jest planowanie procesu nauki. Harmonogram nauki wymaga zachowania ścisłej kolejności niektórych działań. Na przykład nauka teorii nie może się rozpocząć bez wcześniejszego zebrania notatek i zakupu podręcznika. Jednakże same notatki i podręcznik można załatwić w dowolnej kolejności.

Zależności te obrazuje poniższy graf:



Wynik działania programu będzie wyglądał następująco (algorytm Kahna):

Kolejnosc wykonywania zadan:

1. Wybór_przedmiotu
2. Zebranie_notatek
3. Zakup_podrecznika
4. Nauka_teorii
5. Rozwiazywanie_zadan
6. Powtorka_materialu
7. Egzamin

Natomiast dla algorytmu DFS, otrzymamy:

Kolejnosć wykonywania zadań:

1. Wybor_przedmiotu
2. Zakup_podrecznika
3. Zebranie_notatek
4. Nauka_teorii
5. Rozwiazywanie_zadan
6. Powtorka_materialu
7. Egzamin

Ten przykład doskonale obrazuje, jak algorytmy grafowe radzą sobie z węzłami o stopniu wejściowym większym niż 1. Zadanie „Nauka teorii” mogło zostać odblokowane i dodane do kolejki dopiero w momencie, gdy z systemu usunięto zarówno krawędź od strony notatek, jak i od strony podręcznika.

8 Podsumowanie

W projekcie przedstawiono sposób **szeregowania zadań**, który cechuje się prostotą implementacji, możliwością wykrywania cykli oraz szerokim zastosowaniem praktycznym.

Metoda posiada również pewne ograniczenia jakimi są działanie wyłącznie na grafach acyklicznych, brak uwzględniania czasu wykonywania zadań oraz braku wyznaczania harmonogramu dla wielu procesów jednocześnie.